

Consegna 5b - OpenMP

Parallelizzazione di HEAT (`omp_heat.c`)

L'obiettivo dell'esercizio è la parallelizzazione tramite la libreria OpenMP di `omp_heat.c`. Dalle analisi di profilazione precedentemente condotte, la funzione `Jacobi_Iterator_CPU` assorbe oltre il 78% del tempo di calcolo totale. Se applicassimo la direttiva `#pragma omp parallel for` direttamente all'interno della funzione, avremmo aperto e chiuso la regione parallela a ogni singolo step del ciclo temporale (`MAX_ITER`). Questo approccio avrebbe introdotto molto overhead di fork-join costringendo il sistema operativo a creare, sincronizzare e distruggere continuamente i thread, degradando drasticamente le prestazioni.

Per massimizzare l'efficienza, è stata implementata un'unica grande regione parallela posizionata all'interno del `main`, che racchiude l'intero ciclo delle iterazioni temporali (`MAX_ITER`):

- **Gestione delle Sezioni Sequenziali:** all'interno del ciclo `for`, lo swap dei puntatori delle matrici e l'aggiornamento delle condizioni al contorno devono essere eseguiti in modo strettamente sequenziale per evitare *data race*. Per fare questo è stata inserita la direttiva `#pragma omp single`. La barriera di sincronizzazione implicita alla fine del blocco `single` garantisce che nessun thread inizi il calcolo di Jacobi prima che la griglia sia stata correttamente inizializzata e aggiornata per lo step corrente.
- **Decomposizione di Dominio 1D:** la parallelizzazione del calcolo avviene sfruttando il costrutto di work-sharing `#pragma omp for schedule(static)` inserito sul ciclo esterno della funzione. Questo consente ai thread pre-allocati nel `main` di dividersi blocchi contigui di righe della matrice. Ogni core carica linee di cache consecutive, riducendo al minimo i cache miss ed eliminando il rischio di *false sharing*.

Output

Locale:

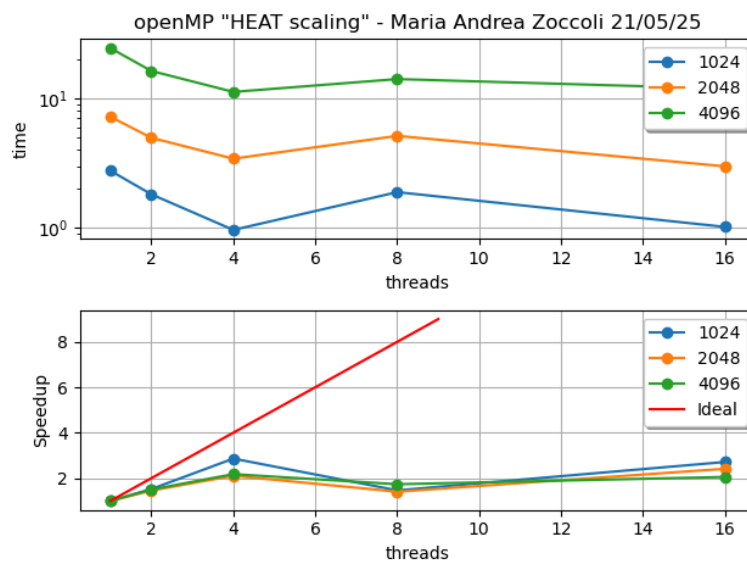
	nt	r	c	iter	t	tj	hostname
OMP	1	2048	2048	500	7.204	6.962	andrea
OMP	2	2048	2048	500	4.959	4.739	andrea
OMP	4	2048	2048	500	3.422	3.138	andrea
OMP	8	2048	2048	500	5.131	4.045	andrea
OMP	16	2048	2048	500	2.986	2.585	andrea

Cluster:

	nt	r	c	iter	t	tj	hostname
OMP	1	2048	2048	500	4.113	4.063	wn20
OMP	2	2048	2048	500	2.191	2.093	wn20
OMP	4	2048	2048	500	1.200	1.080	wn20
OMP	8	2048	2048	500	0.800	0.690	wn20
OMP	16	2048	2048	500	0.701	0.583	wn20
OMP	32	2048	2048	500	0.629	0.504	wn20

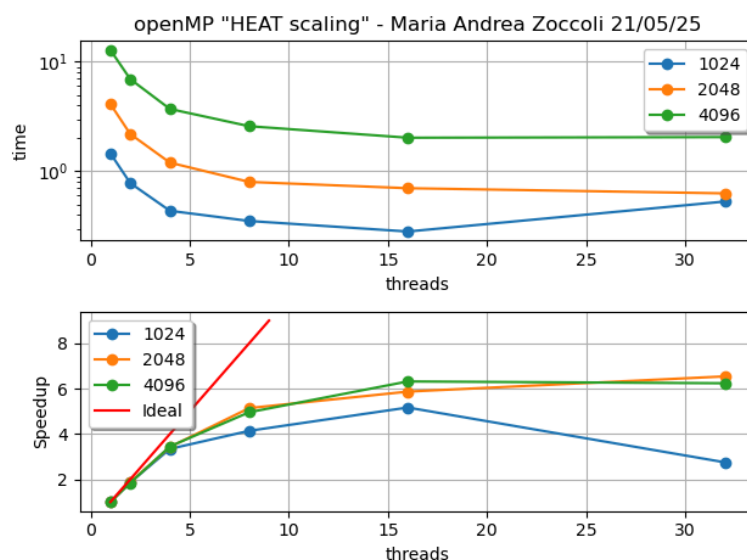
All'aumentare della dimensione della griglia e dei thread, il tempo speso nell'iteratore di Jacobi tende a occupare la quasi totalità del tempo di esecuzione globale, confermando che l'hotspot è stato parallelizzato con successo e che l'overhead introdotto dalle sezioni `single` è trascurabile.

Grafico 1: PC Locale (`omp_heat_scaling_local.png`)



- **Da 1 a 4 thread:** si osserva una decrescita del tempo e un incremento dello speedup quasi lineare. Grazie ai 4 core fisici reali e indipendenti della CPU la pipeline di calcolo viene saturata in modo ottimale.
- **A 8 thread:** appena il parallelismo costringe a utilizzare i core logici dell'hyper-threading, la curva subisce un drastico innalzamento del tempo e un conseguente peggioramento delle prestazioni. L'hyper-threading duplica solo i registri di stato ma condivide le stesse unità funzionali del core fisico, trovandosi le FPU già saturate dal calcolo dei primi 4 thread, i thread logici rimangono in coda, generando conflitti di scheduling e stalli nell'elaborazione.
- **Oltre gli 8 thread:** a 16 thread si nota un andamento frastagliato e inefficiente causato dall'eccessivo overhead di *context-switching* generato dallo scheduler del sistema operativo Host che tenta di emulare un livello di parallelismo superiore a quello hardware disponibile.

Grafico 2: Cluster (`omp_heat_scaling_cluster.png`)



- **Scaling Lineare (1-16 thread):** il tempo scende in modo pulito e costante, poiché la partizione mette a disposizione core fisici reali e indipendenti senza Hyper-Threading attivo, ogni thread OpenMP viene mappato direttamente su una risorsa hardware dedicata ed esclusiva. I thread lavorano sui propri registri e cache locali senza colli di bottiglia architetturali o contese hardware.
- **Saturazione a 32 Thread:** Nel passaggio finale da 16 a 32 thread, lo speedup smette di crescere linearmente e si stabilizza (il tempo a 4096 si attesta a 2.053 s). Questa stagnazione è legata a due fattori concorrenti:
 - **Saturazione del Bus di Memoria:** quando 32 core richiedono simultaneamente dati dalla RAM per aggiornare le righe della matrice, il bus di sistema si satura, costringendo i core a lunghi stati di idle in attesa dei dati.
 - **Impatto della Legge di Amdahl:** man mano che la frazione parallelizzabile viene ridotta, la frazione sequenziale critica racchiusa nelle direttive `#pragma omp single` comincia a pesare proporzionalmente sul tempo totale, dettando il limite asintotico superiore dello speedup teorico

Parallelizzazione e Bilanciamento di Factorize

La fattorizzazione RSA tramite attacco *brute-force* è un problema a natura non deterministica. Lo spazio di ricerca (le possibili chiavi) viene suddiviso in sottomultipli, i "blocchi". Poiché la posizione del fattore primo incognito non è noto a priori, è impossibile prevedere quale blocco richiederà l'intera elaborazione e quale conterrà la soluzione.

L'OMP è stato implementato facendo due scelte architetturali:

1. **Load balancing dinamico:** una decomposizione statica avrebbe causato un grave sbilanciamento del carico, se il thread 0 avesse individuato subito la chiave, gli altri thread avrebbero continuato a processare inutilmente i blocchi assegnati a inizio esecuzione. Mentre, con una distribuzione dinamica a grana fine, i blocchi vengono inseriti in una coda di task condivisa; ogni thread preleva e analizza un blocco alla volta, richiedendone uno nuovo solo al completamento del precedente, mantenendo le risorse computazionali costantemente bilanciate.
2. **Early termination e sezioni critiche:** è essenziale interrompere l'esecuzione di tutti i thread non appena la soluzione viene individuata. È stata introdotta una variabile globale condivisa (`found`), che viene letta all'inizio di ogni iterazione asincrona. Se un thread individua il fattore, alza il flag effettuando una scrittura protetta per evitare *race conditions*. Inoltre, l'aggiornamento del puntatore globale alla chiave trovata (`F=BN_dup(BLOCK_IDX)`) all'interno della funzione di calcolo è stato racchiuso in un blocco `#pragma omp critical` per prevenire errori di segmentazione della libreria OpenSSL nel raro caso di ritrovamenti simultanei.

Output

PC Locale

```
andre@andrea:factorize$ ./omp_factorize -m B81915BC0A2222F4B -a 4

# Modulus: B81915BC0A2222F4B 68 bits - prime: address 4 bit, block 30 bit, num_threads
8

thread 7 processing block 11
thread 0 processing block 8
thread 3 processing block 13
thread 4 processing block 14
thread 6 processing block 12
```

```

thread 1 processing block 15
thread 2 processing block 10
thread 5 processing block 9
thread 3 processed block 13 in 34.9 sec. with result 1
thread 7 processed block 11 in 115.1 sec. with result 0
thread 4 processed block 14 in 115.3 sec. with result 0
thread 0 processed block 8 in 115.3 sec. with result 0
thread 2 processed block 10 in 115.4 sec. with result 0
thread 5 processed block 9 in 116.0 sec. with result 0
thread 6 processed block 12 in 116.1 sec. with result 0
thread 1 processed block 15 in 116.2 sec. with result 0
FOUND: 3534C0B09

```

Stime Matematiche

Stima dei Tempi di Calcolo (limite 24 ore)

La partizione HPC `cpu_guest` impone un wall-time massimo di 24 ore (86.400 secondi):

- **Tempo medio per blocco (26 bit):** $T_{block} = 5.5$ s
- **Numero di chiavi in un blocco da 26 bit:** 2^{26}
- **Numero massimo di blocchi esplorabili in 24h da un singolo core:**

$$N_{24h} = \frac{86400 \text{ s}}{5.5 \text{ s/blocco}} \approx 15.709 \text{ blocchi}$$

- **Dimensione massima in bit di un blocco elaborabile in 24h:**

Poiché $15.709 \approx 2^{13.94}$, lo spazio totale esplorato in 24 ore è $2^{26} \times 2^{13.94} = 2^{39.94}$ chiavi.

⇒ La dimensione massima di un blocco gestibile da un singolo core nel limite di tempo imposto dallo scheduler è di **40 bit**.

Fattorizzazione di un Modulo a 128 bit

Un modulo RSA a 128 bit è il prodotto di due numeri primi grandi al massimo 64 bit ciascuno. L'intero spazio di ricerca utile per garantire di intercettare il fattore più piccolo è dunque di 2^{64} possibilità.

Sfruttando i blocchi della dimensione limite massima calcolata ($B_{max} = 40$ bit), il numero totale di blocchi necessari per coprire l'intero spazio di ricerca sarà:

$$\text{Numero di blocchi} = \frac{2^{64}}{2^{40}} = 2^{24} \text{ blocchi}$$

Fattibilità della Tecnica di Check-Pointing

Il check-pointing è fondamentale per aggirare i time-limit, salvando lo stato del sistema per riprendere il calcolo nei successivi run:

- **Fattibilità per factorize:** elevata ed efficiente, lo stato di avanzamento è descritto da un solo dato, ovvero l'indice del blocco elaborato. Scrivere questo intero su un file comporta un I/O nullo, rendendo il check-pointing un'operazione leggera e sicura.

- **Fattibilità per heat:** possibile ma limitata a causa dell'overhead di I/O, il salvataggio richiederebbe la scrittura su disco dell'intera matrice bidimensionale dei valori in virgola mobile e dell'indice temporale. Per griglie molto grandi, questo genererebbe continui colli di bottiglia (I/O bound). Per essere fattibile, il check-pointing non deve avvenire a ogni iterazione, ma a intervalli molto ampi.